IndiCare

Mobile IoT monitoring

Technical report and evaluation

Synthetic health classification and
consent based caregiver communication

Team members	Nathan Ratichon	2362789
	Guannan Ge	2364229
	Jose Manuel Cortes Ceron	2686162
	Cheng Zhang	2361292
Tutor	Matilda Isaac	

Report date	26 October 2026
Repository	`https://github.com/deepdevjose/Older-Monitoring-Wearable`

Educational use only
This prototype must not be used for diagnosis, treatment or clinical decision making. Evaluation uses synthetic fixtures. Results distinguish server experiments, Android interface tests and hardware observations.

Contents

1 Purpose and assessment scope **1**

2 Requirements and system architecture **1**

3 Hardware and data boundaries **3**

4 Mobile behaviour and programming structure **4**

5 REST integration storage and alert lifecycle **5**

6 Synthetic classification and model evaluation **6**

7 Security configuration and development evidence **10**

8 Testing measured outcomes and improvement **10**

9 Conclusion **11**

References **12**

A API contract and synthetic example **13**

B Reproduction installation and scenario evidence **14**

C Historical development images **17**

D Project team and development history **18**

E Demonstration protocol and evaluation boundaries **19**

1 Purpose and assessment scope

IndiCare examines how an Android gateway, a local REST service and introductory machine learning support an educational monitoring workflow. The application is named CareMonitor 1.0.0. Its evaluated capabilities include persistent synthetic alert episodes, consent based access, a reproducible classification service and native Android account navigation. The evidence combines software tests, isolated HTTPS integration, installation on a Samsung phone and targeted interface instrumentation. Each result is attributed to the layer actually exercised.

The assessment covers patient and caregiver views, three class predictions, confidence, storage, alerts and measurable improvement. Mobile functionality and programming architecture carry 45% of the rubric, making the distinction between server capabilities and visible application behaviour central to the evaluation. This report maps requirements to code, presents quantitative results and analyses the implemented system's coverage. The assessment uses the current versioned API and release records rather than treating historical development captures as proof of current behaviour.[1, 2]

Classification uses synthetic educational labels. Participant measurements, clinical records and induced falls are outside this study. Historical screenshots document technical development without presenting personal physiological values. A separate SisFall experiment is excluded from the reported classification results. The evaluated CSV is identified as a repository dataset; institutional approval or equivalence to a module supplied dataset is not inferred from its presence. This preserves the distinction between reproducibility, assessment provenance and clinical validity.

2 Requirements and system architecture

The primary actors are an adult account, corresponding to the rubric's patient role, and an explicitly authorised caregiver. An adult can approve several caregivers and revoke their access independently. Requirements follow a reading through provenance, validation, transfer, persistence, presentation and alert review. Table 1 associates these behaviours with implementation locations and the evidence available for each one.

Figure 1 distinguishes two implemented server paths. Synthetic snapshots enter the readings service and exercise transparent alert rules, persistent history and private notification inboxes. Synthetic feature vectors enter the classification endpoint and produce a model label and confidence. These services operate independently: the Android care interface retrieves readings and alerts, while its Activity does not consume the classification endpoint. Classification results therefore establish API behaviour, without establishing prediction or confidence presentation on the phone.

A local FastAPI service keeps Python validation, models and account authorisation together. Central inference avoids distributing model updates to every phone but depends on server availability. SQLite offers transactional episode creation with modest deployment complexity. Its single writer architecture suits the evaluated scale; the experiments do not establish large deployment capacity. Native Android Java provides Bluetooth and foreground service access without introducing a cloud service. This describes the technical choice without inferring administrative approval of a particular framework.

The optional ESP32 path supplies device diagnostics over Bluetooth Serial Port Profile to the Android gateway. Source labels distinguish diagnostics from synthetic assessment data. Server inference rejects device sources because synthetic body temperature is not interchangeable with the available surface temperature measurement. HTTPS protects phone to server communication; Bluetooth pairing forms a separate trust boundary. Account and resource authorisation apply independently of either transport.

Observable invariants guide evaluation. A retry preserves one record per sample, and sustained abnormal input preserves one active episode per signal. Unauthorised requests disclose no patient data, while unavailable predictions retain an explicit unknown or error state. Accessibility evidence covers enlarged registration text and control geometry on the installed phone. Timing results are reported as measured

baselines because the supplied specification defines no numeric response or delivery threshold. A measured value is consequently distinguished from a formally satisfied timing requirement.

Table 1. Core requirements traced to current implementation and evidence. S means source inspected, E means backend or JVM software executed, and D means Android UI executed on a physical phone.

ID	Requirement and component	Verification and scope	State
R1	Older adult and caregiver views; MainActivity	Role selection, navigation and caregiver wearable guard checked on device	E/D
R2	Reading storage and retrieval; readings API, SQLite	Idempotency, access and ordering tests pass	E
R3	Reading history and charts; history API, SensorChartView	History endpoint tested; chart component exists outside current care navigation	S/E
R4	Three classes and confidence; ModelService	Two saved models and authenticated inference tested; output available through API only	E
R5	Stored alerts and review; alerts, private inbox	Sustained episode, recovery and independent caregiver review tested	E
R6	Missing, invalid and disconnected states	Validation and relay policies tested; physical outage recovery outside this campaign	S/E
R7	Roles, consent and credentials	Authentication, approved membership, TLS and revocation tested	E
R8	Modular app and accessible navigation	JVM logic, labelled tabs and registration at 200% font scale checked; Activity coupling remains	E/D
R9	Controlled development and contribution	Four main history commits establish repository-level development history	S
R10	Synthetic demonstration and improvement	Synthetic API episodes and backend comparison recorded; care UI excludes synthetic controls	S/E

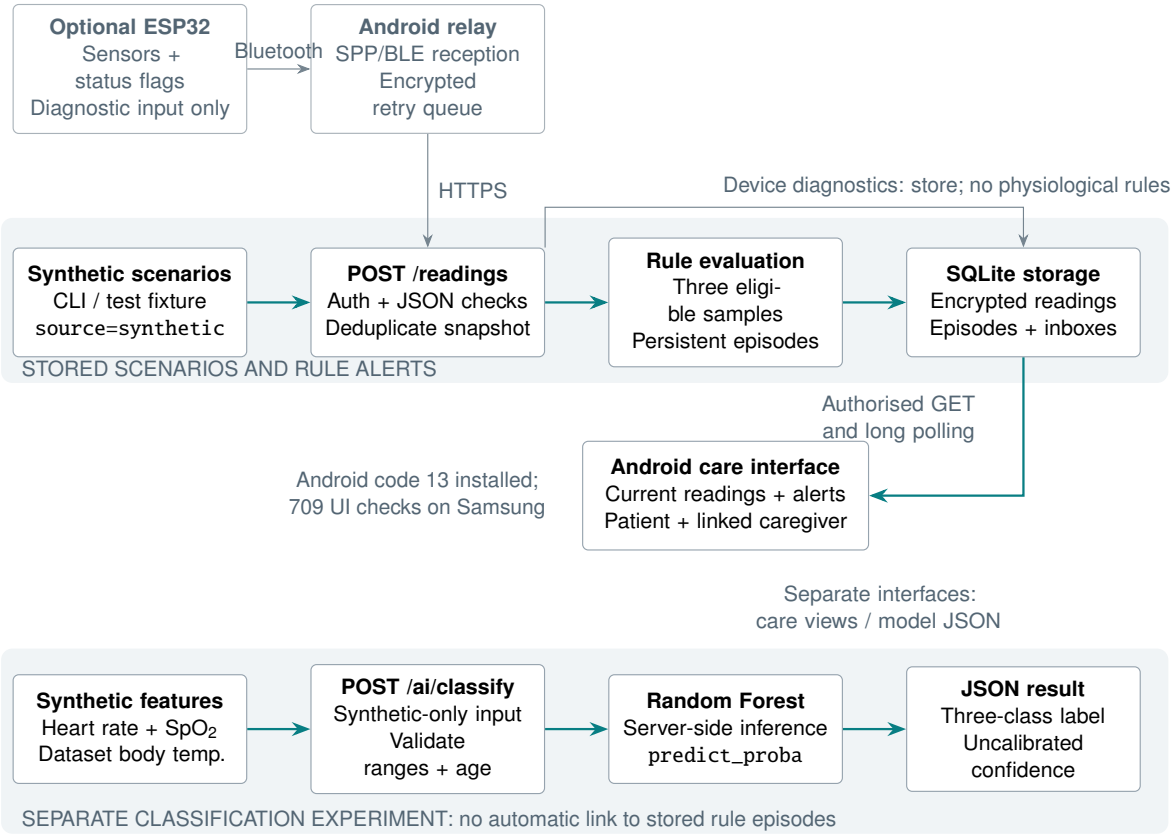


Figure 1. Current logical architecture. Solid teal paths describe implemented server and mobile-source responsibilities; grey paths describe the optional device diagnostic contract. The care interface retrieves readings and alerts, while the synthetic classifier exposes a separate JSON interface. The recorded phone campaign exercises native navigation and registration.

3 Hardware and data boundaries

The wearable source integrates an ESP32, MAX30102 optical module, MPU6050 motion module, GY906 temperature module and photoresistor. Sensor status accompanies snapshots so that missing or warming signals need not become plausible numeric measurements. The light channel is an uncalibrated environmental indicator; darkness alone does not establish wear. Surface temperature cannot establish fever, and the installed modules do not measure blood pressure or hydration.

The optical pipeline uses a rolling sample window and validity flags. These are signal handling choices rather than evidence of physiological accuracy. The recorded USB diagnostic contains 25 JSON messages at approximately one message per second, with optical no contact, the motion module offline and the temperature module online. It retains no physiological values. Firmware compilation and this diagnostic observation are separate evidence: neither identifies the installed binary by a verified hash.[2]

Figure 2 presents the supplied photograph of the physical ESP32 assembly, modules and wiring. It documents the assembled hardware and complements the functional architecture diagram. Visible components and connections alone do not demonstrate sensor calibration, firmware identity or successful data delivery. The historical serial and HTTP captures in Appendix C similarly establish development activity under their displayed versions. Hardware is an extension of the synthetic monitoring study, whose classification results are evaluated separately from physical measurements.

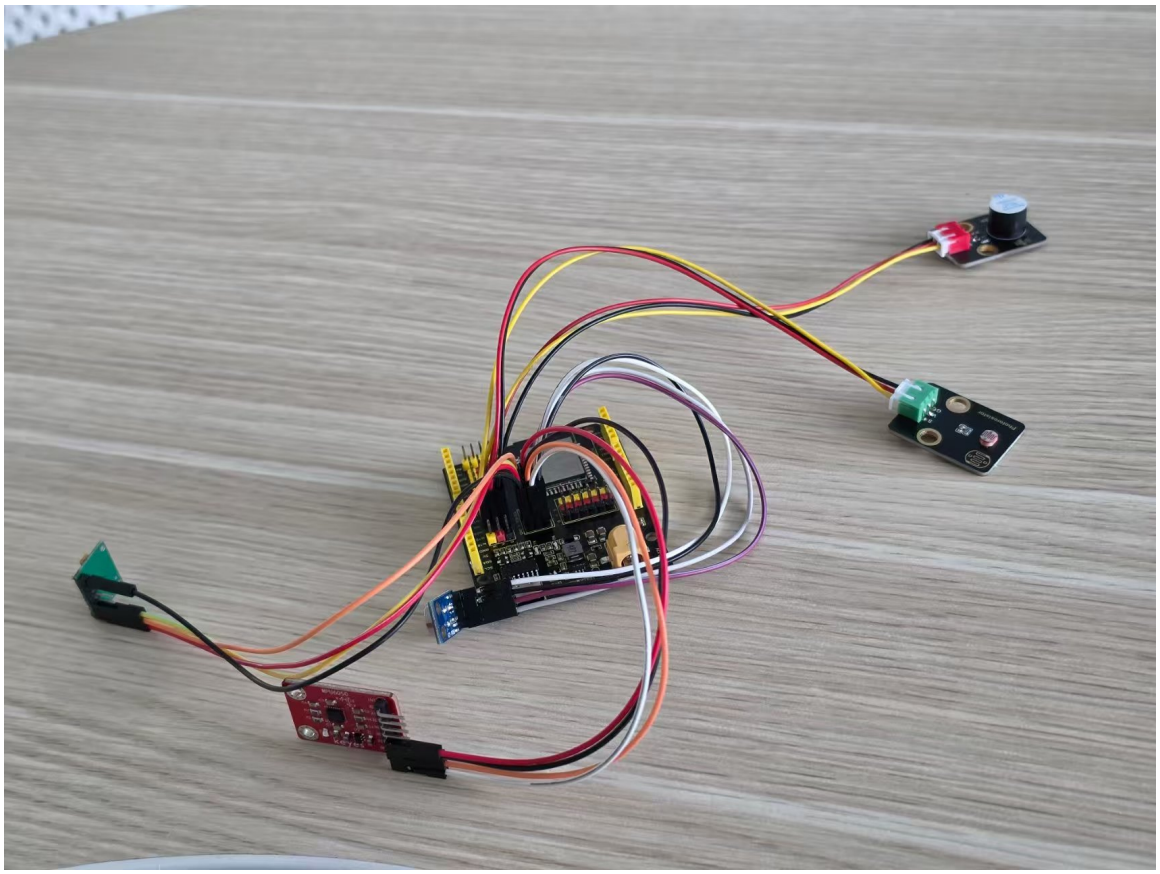


Figure 2. Physical prototype assembly supplied by the team. The ESP32 carrier is central, with jumper wiring to surrounding modules; the buzzer and light sensor boards are visible at the upper right and right. The photograph records the physical arrangement. Module runtime observations are evaluated separately.[6]

4 Mobile behaviour and programming structure

The Android application implements registration, login, adult and caregiver navigation, reading summaries, alert history and settings. Registration introduces the role before collecting name, email and password in four progressive steps. Server configuration belongs to authenticated Settings, keeping technical connection details outside the initial account form. Wearable setup is restricted to the adult role; caregivers use the relationship request and approval workflow. Separate accounts and server enforced revocation prevent shared credentials or locally hidden controls from becoming the access model.

Progressive registration presents one category of information at a time and preserves the draft when the user moves backwards. The primary action stays available while the inputs scroll, making the current step and its completion control identifiable at enlarged text. Password visibility is an explicit user choice, with masking enabled initially. These are concrete interaction decisions supported by instrumentation, rather than a claim of measured preference or reduced cognitive burden. Role selection also determines the subsequent task: the adult configures a wearable, while the caregiver seeks authorised access to another account.

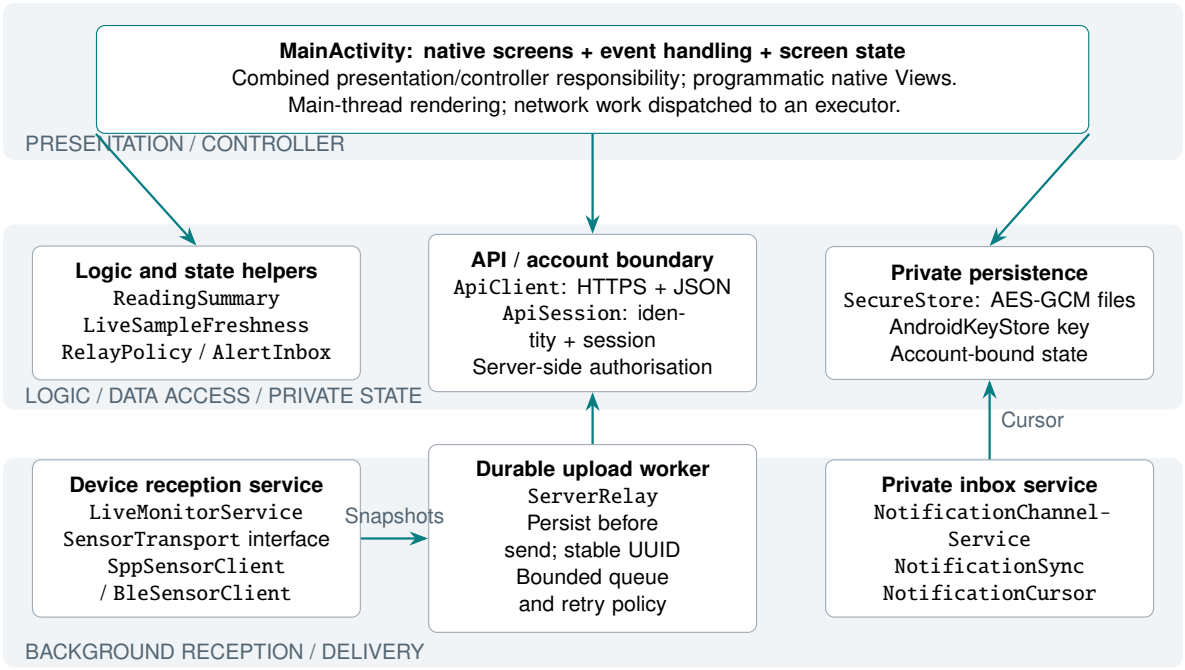
The interface combines labelled cards, text status, icons and large controls. A Samsung SM-S948B running Android 17 hosted version 1.0.0, code 13. Its instrumentation passed 709 checks, including role selection, preserved registration drafts, password masking, navigation and authenticated connection settings. Registration forms were exercised with per Activity font scale 2.0, scrollable inputs and a visible fixed primary action with targets of at least 56 dp. These observations support that specific layout claim; they do not establish a manual TalkBack evaluation or every screen at enlarged text.[14, 5]

Figure 3 shows a layered, MVC inspired arrangement. MainActivity constructs views and coordinates state, combining presentation control with view composition. ReadingSummary, LiveSampleFreshness and

AlertInbox encapsulate behaviour testable outside Android. ApiClient and ApiSession manage requests and account state, SecureStore protects persistence, and SensorTransport separates Bluetooth implementations. This is partial separation of concerns rather than a complete MVVM architecture. Android's guidance provides a useful basis for assessing the boundary between interface orchestration and reusable data responsibilities.[13]

Bluetooth acquisition and queued relay use dedicated services and workers. Network tasks run through an executor, with interface changes posted to the main thread. Session and lifecycle checks prevent callbacks from an earlier screen or account from becoming current state. Notification service activation uses an explicit user action and visible foreground notification. The interface instrumentation preserved permissions, preferences, session and outbox state and created no server accounts, separating controlled UI checks from a real authentication or Bluetooth campaign.

The reading history endpoint and SensorChartView exist, but the Activity neither requests that history nor mounts the chart. The classification API likewise returns three classes and confidence without a corresponding mobile prediction widget. These are precise coverage boundaries for the rubric's mobile history and inference criteria. The installed application's verified navigation and registration behaviour are assessed alongside those boundaries, without promoting compilation or instrumentation into complete sensor to caregiver operation.



Selected source responsibilities and dependencies; a lay-
ered, MVC-inspired structure with combined View/controller work.
SensorChartView exists but is not mounted in the active UI. No complete MVVM implementation is claimed.

Figure 3. Actual mobile organization rather than a proposed MVVM architecture. Transport, HTTPS access, private storage and state helpers are separated from the activity, while the activity retains presentation and controller duties. Source-level separation and JVM checks do not establish Android lifecycle, accessibility, Bluetooth or background-delivery acceptance on a physical phone.

5 REST integration storage and alert lifecycle

The API uses the prefix `/api/v1`. Users authenticate through account endpoints and send a Bearer session token. Readings carry a stable sample identifier, capture timestamp, source and structured payload. Pydantic contracts reject malformed or unsupported inputs, while account and resource checks enforce access. Appendix A records endpoint details and an observed synthetic prediction, making the current contract

Table 2. Role behaviour in the current release. Server checks enforce access; the Android UI also restricts wearable controls to the older adult.

Action	Older adult	Approved caregiver
Read current overview and alert history	Own account	Linked adults only
Submit own sensor snapshots	Own account	Cannot impersonate adult
Approve or revoke caregiver	Own circle	Cannot self approve
Review alert	View own history	Own acknowledgement only
Account and connection settings	After sign in	After sign in
Guided wearable connection	Own wearable only	Controls unavailable
Reading chart navigation	Outside care navigation	Outside care navigation
AI label and confidence	API only	API only
Registration accessibility	200% layout checked	200% layout checked

explicit.[3]

The phone queue retains snapshots during temporary upload failure. Reusing a sample identifier makes a retry idempotent: a lost response creates neither another sample nor another alert. The server writes reading state, alert state and private notification entries transactionally. SQLite isolation lets readers observe committed changes; it does not guarantee delivery to a disconnected handset.[16] Figure 4 traces a qualifying upload through storage and fanout to two approved caregivers.

The outbox stores each item before transmission and removes it after acknowledgement. Its capacity is two hundred items, so the design does not imply lossless indefinite disconnection. Items retain account and endpoint ownership to prevent relay across sessions. Authentication, permission and validation errors pause sending; transient failures use retry and backoff. Policy tests exercise these decisions. Actual process termination, AndroidKeyStore persistence and subsequent physical delivery form a different device execution boundary from those tests.

Synthetic rule episodes require three recent valid samples with distinct timestamps separated by one to thirty seconds. Sustained abnormal input creates one unresolved episode per signal. Further abnormal samples update its evidence without repeatedly creating the same episode. Three eligible observations no longer triggering the rule recover it. Missing contact, disconnected sensors and invalid values cannot close an episode because unusable evidence does not establish recovery. Figure 5 makes these transitions explicit.

Acknowledgement records caregiver specific review rather than recovery. One caregiver's action does not suppress another caregiver's history. Delivery acknowledgement is also distinct from review. A persistent private inbox and authenticated long polling support recovery after interruption, while the reported timing experiments measure server processing rather than Android notification arrival.

The synchronous classification route runs through FastAPI's thread pool.[11] Two bounded inference slots and a per account request limit constrain work. A successfully loaded model is cached; an unavailable or busy model produces an explicit error instead of a fabricated normal prediction. JSON inference responds without creating a stored reading or alert. Model confidence therefore does not drive the rule episodes evaluated in this report.

6 Synthetic classification and model evaluation

The experiment reads the 60,000 row repository CSV using pandas. It retains three numeric features: synthetic heart rate, oxygen saturation and body temperature. Identifiers, other measurements and alert columns are excluded from the feature matrix. Missing, nonnumeric or nonfinite values fail preparation instead of being interpreted as healthy. Targets count abnormal flags across the three alert columns: zero

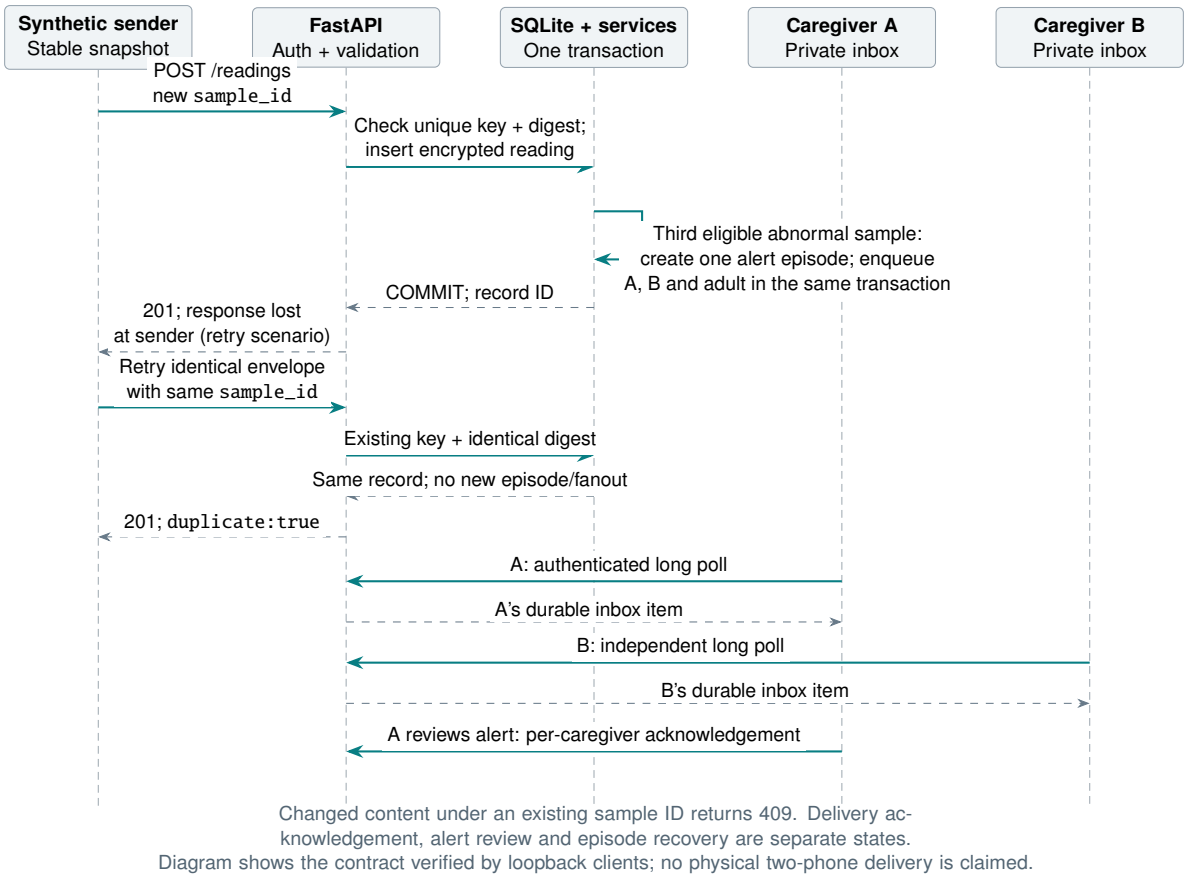
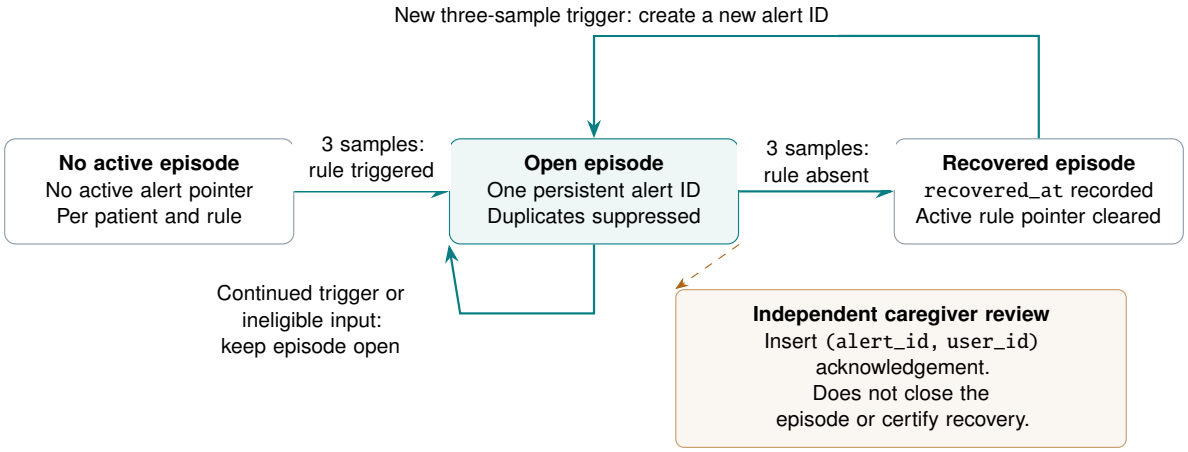


Figure 4. Reception, idempotent retry and two-recipient fanout. A new reading, any resulting rule alert and recipient inbox entries commit atomically. Replaying the same envelope returns the existing record and does not repeat alert creation. Caregivers retrieve separate inbox items; one caregiver's review does not mark another caregiver's alert as reviewed.



Eligibility is a gate, not another clinical state. The latest three readings must be synthetic, fresh within the supported two-minute window, ordered as the latest capture, and separated by 1–30 seconds. Rule-specific validity/contact checks apply. Recovery requires valid non-triggering observations of that rule; stale, missing, invalid or device-diagnostic input cannot establish recovery.

Figure 5. Implemented educational episode lifecycle for one patient–rule pair, derived from `alerts.py` and the SQLite schema. The recovered alert remains in history while its active pointer is cleared. "Recovered" describes the synthetic rule episode; it is not a health outcome. Caregiver review is an independent per-user record.

gives Normal, one Warning, and two or three Critical. These constructed educational categories have no independent clinical ground truth.[4]

Repeated feature tuples could place identical examples in training and evaluation. The training script groups the 18,939 unique feature tuples before a stratified split with seed 42. Every occurrence of one tuple stays in one partition. The resulting counts are 36,090 training, 12,143 validation and 11,767 test rows. This controls exact tuple leakage without creating external validation. The test partition is a local holdout, not an assessor controlled hidden test set.

Two introductory supervised classifiers are compared. Logistic regression uses StandardScaler fitted within its training pipeline, balanced class weights and at most 2,000 iterations. Random Forest uses 120 trees, maximum depth fourteen, minimum leaf size two and balanced class weights. Validation macro F1 selects the model, with Critical recall as tie breaker. Test results are reported after selection rather than used to select. Random Forest reaches validation macro F1 of 1.0000 against logistic regression's 0.7534 and is saved for endpoint inference.

Table 3 reports accuracy and macro precision, recall and F1. Macro averaging weights categories equally; Critical recall also exposes missed critical classifications.[12] Figure 7 presents both confusion matrices with true classes as rows. Logistic regression misclassifies 905 of 3,857 Critical examples and assigns Warning to 420 of 3,366 Normal examples. Random Forest makes no observed holdout errors. Table 4 defines the corresponding rates and denominators.

The perfect forest result reflects reconstruction of deterministic synthetic boundaries rather than medical reliability. Robustness to another generator, sensor noise or distribution shift is outside this experiment. Confidence is the largest predict_proba score, rounded to six decimals. It is not calibrated against real outcomes and cannot represent the probability of disease or emergency. The inference contract requires fresh synthetic features within recorded training ranges. Before deserialisation, the local model file is hash checked and its scikit learn version compared with metadata. This mechanism does not make untrusted model files safe to load.

Logistic regression offers an interpretable baseline, but linear decision surfaces cannot readily represent every disconnected region created by combinations of threshold flags. A tree ensemble represents those combinations more directly. This interpretation is consistent with the matrices without establishing a unique explanation for each error. Reporting all classes prevents a favourable average from concealing weak Critical recall. The result is a reproducible local model comparison whose synthetic targets and dataset provenance delimit its interpretation.

Evaluation keeps feature preparation, model selection and final reporting distinct. Scaling is learned from training data rather than the complete dataset, and validation guides selection before the holdout is inspected for the reported comparison. Grouping prevents duplicate feature tuples from inflating performance through exact repetition across partitions. It does not remove the shared rule generator behind all three partitions. The resulting high score is therefore informative about reconstruction of the supplied synthetic task, while generalisation to a new population is a separate scientific question.

Table 3. Local validation and test performance. P, R and F1 are macro averages; all values are percentages. Validation selects the model.

Model	Partition	Accuracy	P	R	F1	Critical R
Logistic regression	Validation	75.15	75.18	75.53	75.34	73.85
Random Forest	Validation	100.00	100.00	100.00	100.00	100.00
Logistic regression	Test	75.35	75.85	76.46	76.11	76.54
Random Forest	Test	100.00	100.00	100.00	100.00	100.00

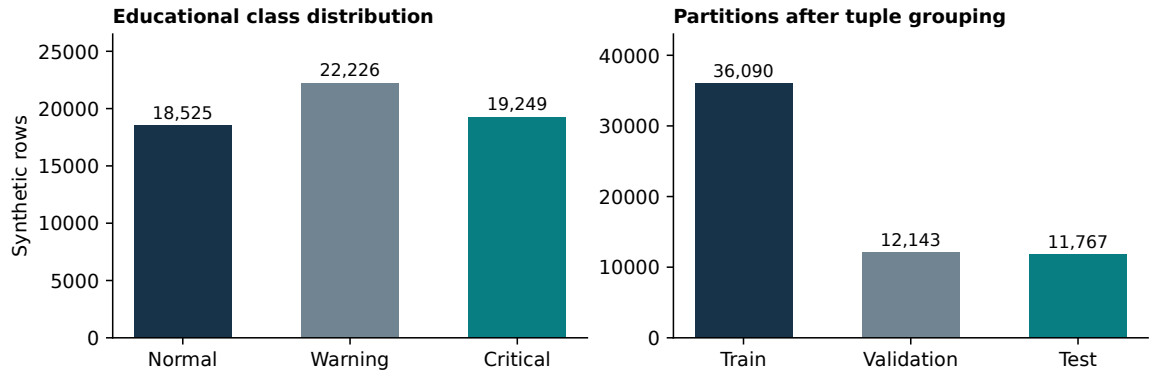


Figure 6. Class balance and row partitions from the saved training metadata. Splitting operates on unique feature tuples, so row proportions differ slightly from 60/20/20. These labels are educational categories.

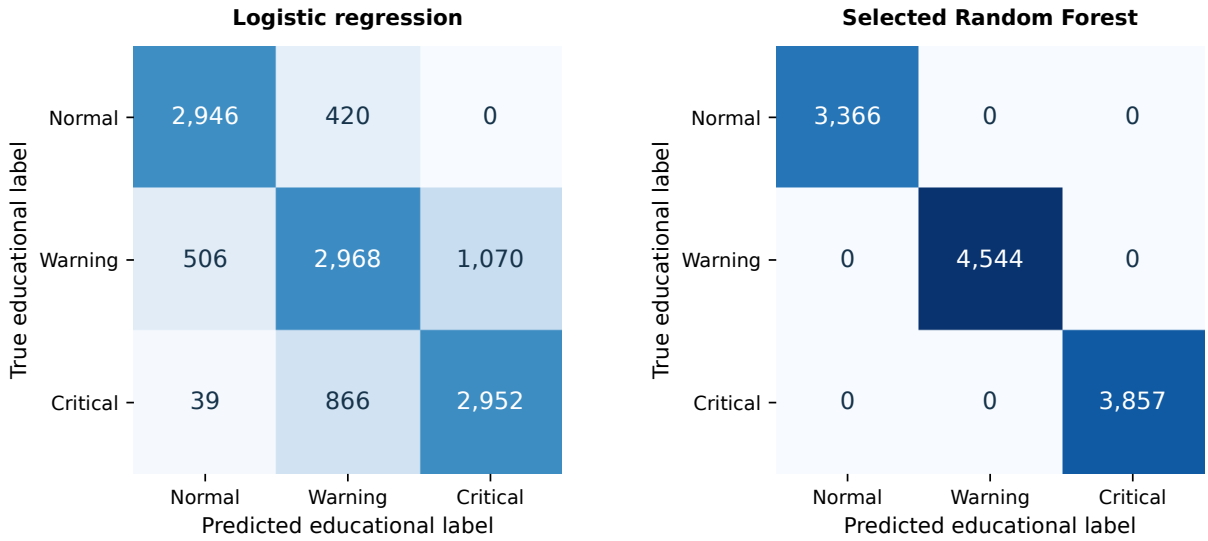


Figure 7. Confusion matrices for 11,767 locally held out synthetic rows. The selected forest reconstructs the rule derived labels perfectly on this split. This is neither clinical validation nor an assessor hidden test.

Table 4. Classification errors and alert observations with explicit denominators. Classifier rates do not measure phone delivery failures.

Measure and denominator	Logistic regression	Random Forest
Missed Critical classification: predicted other class / all true Critical	905/3,857 = 23.46%	0/3,857 = 0%
False Warning on Normal: predicted Warning / all true Normal	420/3,366 = 12.48%	0/3,366 = 0%
False Critical on noncritical: predicted Critical / true Normal or Warning	1,070/7,910 = 13.53%	0/7,910 = 0%
Server episode campaign	20/20 created; 0/20 duplicate episodes	
Missed Critical alert at caregiver phone	Outside server benchmark scope	
False Warning delivered to caregiver	Outside server benchmark scope	

7 Security configuration and development evidence

Accounts use Argon2id password hashes and random session tokens with hashed server representations. Resource access requires the correct role and an active approved relationship to the adult. Invitation codes request access without independently authenticating caregivers or disclosing readings. Expiry, limited attempts and adult approval constrain guessing. Argon2id follows established storage guidance; its inclusion is an implementation property rather than evidence of a comprehensive security audit.[15]

HTTPS verifies certificate names. Sensitive server fields use authenticated AES GCM encryption, while Android protects stored state through its KeyStore backed store. Identifiers, timestamps and event types are visible database metadata. Private configuration, database state and signing material are excluded from version control. The checked in server certificate is public trust material, and backups use a passphrase with restoration invalidating sessions. These distinctions define the protection actually provided.

Repository history records development under one contributor identity. It establishes versioned changes without identifying each student's personal responsibility, controlled peer review or agreed contribution factor. Appendix D presents the supplied four member roster and student IDs alongside the attribution basis. The report makes no individual authorship claim from a shared repository identity and does not infer equal effort from membership alone. This treats collaboration evidence as an attribution question separate from software correctness.[10]

8 Testing measured outcomes and improvement

Current release records report 88 passing Python tests, 2,830 JVM assertions and seventeen passing HTTPS integration checks. Device instrumentation adds 709 targeted Android interface checks. These are different measurement units, reported separately in Table 5. The HTTPS fixture uses fictional accounts, strict hostname verification and two authorised caregivers. It covers revocation, independent review, idempotent retry and model serving. The device campaign tests onboarding and navigation rather than extending that fixture through Bluetooth and the phone network.[5, 7]

A reproducible benchmark uses disposable databases and an in process ASGI TestClient. Twenty five warm classification requests produce mean response time 17.34 milliseconds and p95 19.87 milliseconds. The first request takes 1,059.29 milliseconds, including lazy model loading. Twenty independent synthetic pulse episodes produce p95 9.09 milliseconds for third qualifying ingestion and 15.02 milliseconds from immediately before ingestion until an authenticated inbox read completes. All twenty retry and sustained sample checks retain one unresolved episode.[8]

These measurements exclude TLS, network transport, the preceding sampling interval, long polling and Android rendering. They establish local server processing rather than caregiver alert latency. Table 6 records boundaries and sample counts. Likewise, classifier false warnings and missed Critical labels describe holdout prediction errors, not complete application delivery failures. Keeping these denominators distinct makes both performance and reliability claims interpretable.

The forest improves test accuracy by 24.65 percentage points and macro F1 by 23.89 points over logistic regression. A separate chronological backend experiment compares the initial server at commit 6757c0a with the evaluated implementation. Both receive twenty distinct synthetic samples twice, representing a lost response and retry. The initial server stores forty rows; the evaluated server stores twenty, with every retry reusing its existing identifier. This removes twenty duplicate rows in the matched campaign. The adapter script documents the different contracts. The result demonstrates improved persistence semantics rather than faster operation or mobile inference integration.[9]

Scenario coverage distinguishes deterioration from malformed input. Valid synthetic sequences progress from normal values to sustained abnormal conditions and recovery. Missing fields, nonfinite values, stale timestamps and absent optical contact instead cause rejection or preserve unknown state. Retry policies and lost responses exercise network failure decisions; unlinked and revoked caregivers exercise authorisation.

Appendix B specifies the measurement protocol and separates executable software scenarios from physical transport and operating system behaviour.

The combined evidence supports precise conclusions: server alerts persist without retry duplication, model results are reproducible, and progressive account setup renders on the installed Android release. Physical Bluetooth pairing, complete phone to server delivery, manual screen reader use and mobile prediction display are outside the demonstrated coverage. Those boundaries constrain the rubric interpretation without weakening the observed results or suggesting that one test layer substitutes for another.

Table 5. Executed verification and its scope. Counts are not added because the units differ.

Evidence	Result	Scope and exclusions
Python suite	88/88 tests	Backend contracts, models, CLI and data preparation; current rerun recorded
Mobile JVM suite	2,830 assertions	Pure Java logic; no Android UI or KeyStore runtime
HTTPS integration	17/17 checks	Real loopback TLS, fictional accounts; no phone or Bluetooth
Android release	Installed	APK 1.0.0, version code 13; signing and alignment verified; SHA-256 `417c12ef...206378`
Synthetic benchmark	20/20 episodes	Temporary databases; retries and sustained samples do not duplicate
Android UI on phone	709 checks	Samsung SM-S948B, Android 17 / API 37; role setup, tabs, Settings and registration at 200% font scale. Live Bluetooth and phone–server delivery outside scope

Table 6. Current local server processing measurements in milliseconds. Percentiles use linear interpolation; no transport or Android latency is included.

Timed operation	N	Mean	p50	p95
Warm authenticated classification	25	17.34	16.85	19.87
Third qualifying reading and episode commit	20	7.08	6.95	9.09
Third reading through authenticated inbox GET	20	12.24	12.02	15.02
Cold classification including model load	1	1,059.29	—	—

The first two episode samples were already stored. Inbox timing uses the fictional adult's inbox; caregiver fanout is verified separately by the HTTPS integration fixture.

9 Conclusion

IndiCare demonstrates a local architecture with persistent synthetic alerts, explicit access control, a reproducible two model comparison and verified Android onboarding. Random Forest reproduces the synthetic holdout labels, while idempotent ingestion halves stored rows in the matched retry campaign. The installed application's progressive role based registration is supported by targeted device evidence. Classification confidence is an API capability, and physical transport and clinical effectiveness are outside the demonstrated results. The report's contribution is a traceable educational evaluation that connects each quantitative claim to its actual data, implementation layer and measurement boundary.

References

[1] IOT308TC. *Assessment 2 AI Enabled Mobile IoT Healthcare Monitoring Application*. Supplied assessment brief and marking rubric, `rubrik.docx`. 3,000 words ±10%; eight weighted criteria.

[2] IndiCare project. *CareMonitor 1.0.0 implementation and acceptance status*. Repository commit `5c650c1`, `README.md`, `docs/STATUS.md`, `docs/SECURITY_ETHICS.md` and `release/FIRMWARE_DIAGNOSTIC.json`. `https://github.com/deepdevjose/Older-Monitoring-Wearable`.

[3] IndiCare project. *Versioned API and implementation*. `docs/API.md`, `server/caremonitor/api.py`, `schemas.py`, `alerts.py`, `db.py`, `routes_ml.py` and Android source in `mobile/app/src/main/java/org/caremonitor/demo/`.

[4] IndiCare project. *Synthetic health model training and evaluation*. `tools/train_models.py`; `models/synthetic-health-v1/metadata.json`. Seed 42; scikit learn 1.8.0. Snapshot included as `evidence/model-metadata.json`.

[5] IndiCare project. *Release software validation, Android build and device interface checks*. `evidence/VALIDATION.json`, `ANDROID_BUILD.json`, `ANDROID_UI_EVIDENCE.json`, `pytest-current.xml` and `jvm-current.txt`, 26 October 2026. APK code 13; 709 checks on Samsung SM-S948B with Android 17.

[6] IndiCare team. *Physical prototype assembly photograph*. Supplied 26 October 2026; reproduced as `figures/hardware_prototype.jpg`. Original source path and SHA-256 recorded in `evidence/hardware_photo.json`.

[7] IndiCare project. *Isolated HTTPS integration evidence*. `release/INTEGRATION_EVIDENCE.json`, 25 October 2026. Seventeen checks; fictional accounts; no physical phone.

[8] IndiCare report audit. *Current server processing benchmark*. `tools/benchmark_api.py` and `evidence/benchmark-current.json`, 26 October 2026. In process ASGI, temporary databases; raw samples and boundaries retained.

[9] IndiCare report audit. *Lost response retry comparison*. `tools/benchmark_idempotency.py` and `evidence/idempotency-before-after.json`, 26 October 2026. Initial `6757c0a:main.py` compared with current API using twenty matched synthetic pairs.

[10] IndiCare project. *Main branch version history*. Four commits, 25–26 October 2026; evidence in Appendix D. `https://github.com/deepdevjose/Older-Monitoring-Wearable/commits/`.

[11] FastAPI. *Concurrency and async await*. Official documentation. `https://fastapi.tiangolo.com/async/`. Accessed 26 October 2026.

[12] Scikit learn. *Metrics and scoring quantifying the quality of predictions*. Version 1.8 documentation. `https://scikit-learn.org/1.8/modules/model_evaluation.html`. Accessed 26 October 2026.

[13] Android Developers. *Guide to app architecture*. Official documentation. `https://developer.android.com/topic/architecture`. Accessed 26 October 2026.

[14] Android Developers. *Make apps more accessible Views*. Official documentation. `https://developer.android.com/guide/topics/ui/accessibility/views/apps-views`. Accessed 26 October 2026.

[15] OWASP. *Password Storage Cheat Sheet*. `https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html`. Accessed 26 October 2026.

[16] SQLite. *Isolation in SQLite*. Official documentation. `https://www.sqlite.org/isolation.html`. Accessed 26 October 2026.

A API contract and synthetic example

All paths below use `/api/v1`. A session is required except for registration and login. Exact validation and error bodies are defined by the source, not the historical screenshots.

Table 7. Selected current REST operations.

Method	Path	Purpose and access
POST	`/auth/register`, `/auth/login`	Create an educational account or start a session
GET/POST	`/me`, `/auth/logout`	Read own profile or revoke current session
POST	`/readings`	Adult uploads own source labelled snapshot; idempotent sample ID
GET	`/patients/{id}/overview`	Current reading and care state; own or approved resource
GET	`/patients/{id}/readings`	Persisted reading history; own or approved resource
GET	`/patients/{id}/alerts`	Stored alert episodes; own or approved resource
POST	`/alerts/{id}/acknowledge`	Record the approved caregiver's own review
GET	`/notifications`, `/notifications/wait`	Private durable inbox or authenticated long poll
GET	`/ai/model`	Educational model identity and limitations
POST	`/ai/classify`	Synthetic features to class and confidence; no automatic reading or episode persistence

A.1 Observed inference request and selected response fields

The following normal example is retained in `evidence/benchmark-current.json`. Values are synthetic fixtures. Confidence is a model class score. The complete response also includes the model limitations.

```
POST /api/v1/ai/classify
Authorization: Bearer <session supplied at runtime>

{
  "source": "synthetic",
  "features": {
    "heart_rate_bpm": 75.0,
    "spo2_percent": 98.0,
    "body_temp_c": 36.5,
    "sample_age_seconds": 0.0
  }
}

{
  "classification": "Normal",
  "confidence": 0.999263,
  "model_id": "synthetic-health-v1",
  "source": "synthetic",
  "educational_only": true
}
```

Table 8. Expected failure behaviour of classification and data endpoints.

Condition	Behaviour to demonstrate
Missing or invalid session	Reject protected request; no patient data returned
Wrong patient relationship	Deny resource without revealing its contents
Missing feature, NaN, stale feature vector or device source	Classification rejected; no invented prediction
Same sample ID with changed content	Conflict response; existing sample preserved
Prediction limit exceeded	HTTP 429; user can retry later
Model unavailable or inference slots occupied	HTTP 503; no invented prediction is returned

B Reproduction installation and scenario evidence

B.1 Evidence package and word count

The prose count is generated by `tools/count_words.py`: 2,926 words in explicit `reportprose` environments. Headings and citation markers are excluded; including the short headings would remain within the permitted range. Tables, figures, title, references, test evidence and appendices are excluded as the assessment specifies. No source code listing appears in the main report.

Report evidence	Purpose
`model-metadata.json`	Training recipe, split, class metrics, artifact hashes and limitations
`benchmark-current.json`	All current timing samples and measurement boundaries
`idempotency-before-after.json`	Matched retry campaign with revision IDs and record counts
`pytest-current.xml`	Current 88 test rerun in JUnit format
`jvm-current.txt`	Current 2,830 assertion Java run
`ANDROID_UI_EVIDENCE.json`	709 interface checks on Samsung SM-S948B; registration at text scale 2.0
`hardware_photo.json`	Original hardware photograph, source path, copy identity and scope
Release JSON snapshots	Code 13 build, software validation, HTTPS integration and diagnostic records
`crop_manifest.json`	Source image hashes and exact crop coordinates

B.2 Build and isolated checks

Run the following from the repository root, with its Python environment installed. The benchmark scripts use temporary databases and fictional accounts, not the running server. They write separate rerun files rather than replacing the recorded results. The historic comparison requires Git commit `6757c0a`; it adapts the legacy unversioned API and current versioned API to the same duplicate delivery campaign. It is not a throughput comparison.

```
python3 docs/report/tools/count_words.py
bash docs/report/build.sh
```

```
.venv/bin/python -m pytest tests
.venv/bin/python mobile/scripts/test_reading_summary.py --jdk /usr
.venv/bin/python docs/report/tools/benchmark_api.py
.venv/bin/python docs/report/tools/benchmark_idempotency.py
```

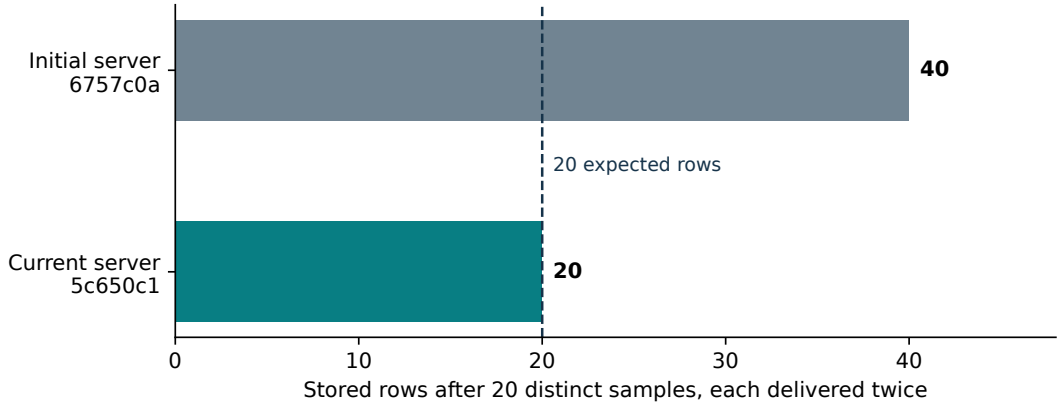


Figure 8. Measured chronological improvement in backend persistence. Forty successful responses represent twenty unique samples delivered twice. The baseline creates twenty extra rows; the current API preserves twenty unique records. The comparison excludes handset and network behaviour.

B.3 Installation and execution procedure

The release APK and source archive are distributed under `release/`. The recorded APK is version 1.0.0, code 13, minimum API 24 and target API 37. Its SHA-256 is `417c12efa634e960b5e3f0ab680a14951b4357a8580f59eb7b7c75caea206378`. Installation and launch on Samsung SM-S948B with Android 17 are recorded alongside 709 native interface checks. The following procedure documents replication of the separate account, server and interface paths. The operational guide is `docs/SERVER.md`.

1. Prepare the Python environment from `requirements.txt`; initialise the local server using `./carectl init -host YOUR_LAN_IP`. Keep generated keys private.
2. Use `./carectl start` and `./carectl status` to check HTTPS service readiness. Ensure the public CA bundled in the APK matches that server. Follow `docs/SERVER.md` when regenerating a build.
3. Install `release/CareMonitor-1.0.0.apk` using Android's authorised local installation flow and retain its hash and device identity. The recorded device uses Android 17/API 37; its interface tests do not grant permissions or activate sensors.
4. Use fictional adult and caregiver accounts for server experiments. Registration presents role, name, email and password as separate steps; the adult issues an invitation and approves the caregiver request. The server enforces that relationship.
5. Run `./carectl demo` with fictional adult credentials. This sends three synthetic rule samples; verify one episode and independent caregiver review. This command does not demonstrate a mobile model confidence view.
6. Exercise classification with fixed synthetic features through `/api/v1/ai/classify`; inspect the returned class, score and source. Reading persistence and retrieval use the separate readings endpoints. Care navigation and model inference have distinct presentation boundaries.
7. Reproduce the interface campaign with `mobile/scripts/test_onboarding_device.py`, selecting the exact device serial. The auxiliary instrumentation checks role selection, back navigation, password masking, enlarged registration forms and authenticated Settings using fictional in-memory profiles.

B.4 Scenario coverage matrix

Table 9. Reproducible scenarios, observed evidence and execution boundaries.

Scenario	Input and expected result	Recorded evidence and scope
Normal	Fixed synthetic normal features; return Normal and score; valid normal snapshots create no rule episode	Model/API execution; classification is a server result, separate from care navigation
Deteriorating	Move from eligible normal snapshots to three distinct abnormal snapshots; one episode after persistence	Sustained condition and recovery test in isolated server fixtures
Critical	Select known Critical CSV fixture; return expected class; independently exercise an abnormal rule sequence	Held out model matrices and server episode campaign; delivery rate is a separate measure
Missing data	Omit required model feature; reject. Missing optical contact must not trigger or recover a rule	Missing feature and invalid contact tests; care views retain explicit unknown states
Invalid data	NaN, unexpected field, unsupported feature or stale age; reject classification	`test_ml.py`, `test_ai_api.py`, request limits tests
Network failure	Lose upload response and retry same ID; retain one row and active episode	Matched backend comparison and relay policies; excludes physical radio interruption
Unauthorised	Query unlinked adult, then revoke previously approved caregiver; disclose no resource	Server IDOR and revocation tests; native caregiver wearable controls are also rejected
Recovery	Three eligible observations no longer meeting the rule; recover episode; later new sustained event may create another	Sustained condition recovery test; review is not recovery

Exact automated anchors in `tests/test_api.py` include `test_sustained_condition_one_episode_valid_recovery_then_new_episode`, `test_idempotency_conflict_and_capture_order`, `test_idor_cannot_read_or_write_other_patients` and `test_revoke_removes_data_and_queued_notifications`. Model input boundaries are exercised by `tests/test_ai_api.py::test_synthetic_api_rejects_device_and_missing_features`. These names locate executable checks; they do not represent installed Android tests.

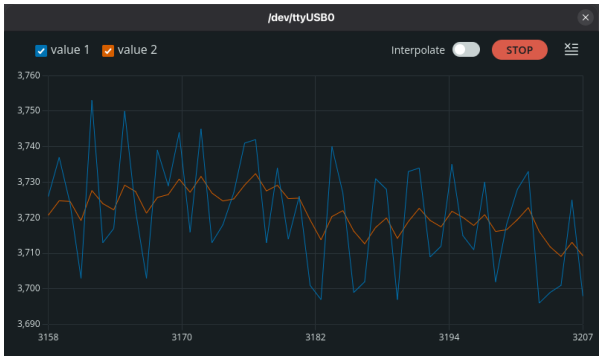
B.5 Application latency and improvement protocol

Select an initial revision and an improved revision before the experiment. Use the same device, dataset fixture IDs, server hardware, network and permissions. After identical warmup, record at least thirty trials per scenario. Measure API time from dispatch to parsed response, and caregiver latency from the first qualifying synthetic input to visible notification. Keep raw timestamps, request IDs, failed trials and timeouts. Report N, mean, p50 and p95 for both revisions; do not silently omit failed deliveries.

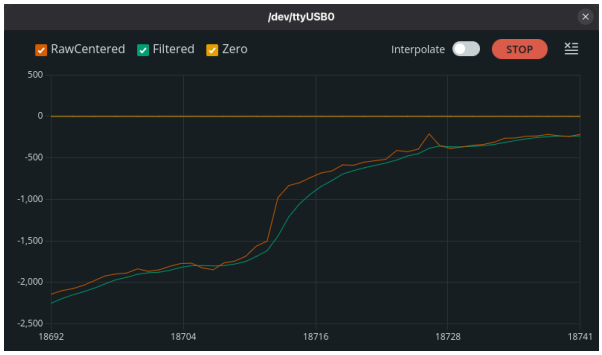
For missed Critical alerts, divide Critical scenarios without a required visible alert within the declared observation window by all Critical scenarios that should generate one. For false Warnings, divide Normal scenarios producing an unwanted Warning by all Normal scenarios. Define the observation window before testing. Model predictions and rule alerts have different semantics; a latency experiment must identify the rule output and its observation window. Separately report duplicates and data loss after reconnection.

C Historical development images

The following supplied captures are cropped for relevance. No output or values have been synthesised. Source paths and crop coordinates are recorded in `evidence/crop_manifest.json`. They document historical software experiments, separately from the assembly in Figure 2 and the Android code 13 interface campaign.



(a) Raw and smoothed ADC traces.



(b) Centred input, filtered output and zero reference.

Figure 9. Serial plotting during development on 13 October 2026. The uncalibrated ADC plots support qualitative signal conditioning observations. Axes do not establish physical time, clinical accuracy or a quantified before and after noise reduction.



Figure 10. Local API availability and successful historical uploads on 13 October 2026. The old `/sensor-data` route returns HTTP 200 while the optical module reports `no_finger`. Successful transport does not establish measurement validity, persistent storage or the final versioned API.

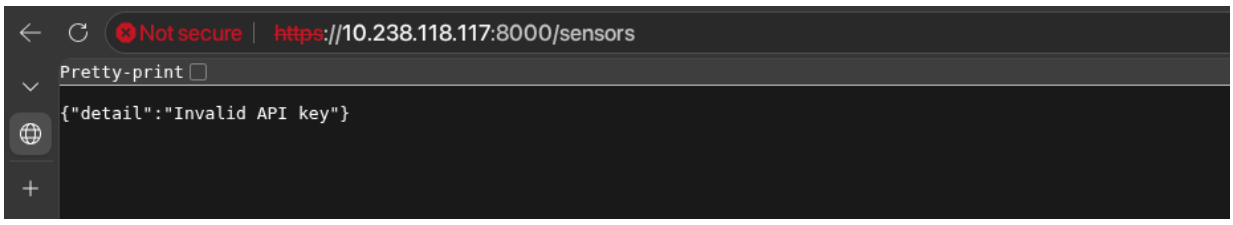


Figure 11. Historical authentication rejection on 15 October 2026. The browser reports an invalid API key and also marks the connection insecure. This establishes a negative authentication path for that version; it is not evidence of trusted HTTPS or the current account based contract.

D Project team and development history

The team roster identifies the academic submission. Repository chronology provides the technical change record; commit metadata is not used to infer a division of personal responsibilities or an individual contribution multiplier.

Team member	Student ID
Nathan Ratichon	2362789
Guannan Ge	2364229
Jose Manuel Cortes Ceron	2686162
Cheng Zhang	2361292

The tutor is Matilda Isaac. The names and IDs are reproduced as supplied by the team and are maintained in `metadata.tex`.

Date	Commit	Recorded subject
25 Oct 2026	`27a72d4`	Initial commit
25 Oct 2026	`6757c0a`	upload files
25 Oct 2026	`7e6c494`	upload csv files and data for ML part
26 Oct 2026	`5c650c1`	update: mayor fixes

The recorded Git identity is José Manuel Cortes Cerón. The four commits establish the baseline chronology. The Android interface campaign and this report revision also include local working-tree changes, identified by the release input hashes rather than represented as additional commits. This preserves the distinction between committed history, supplied documentation and the exact built artefact.

E Demonstration protocol and evaluation boundaries

The demonstration separates three observable layers: native care navigation, synthetic server evaluation and the physical bench assembly. This structure prevents a successful result in one layer from being used as evidence for a different layer.

1. Identify APK 1.0.0/code 13 and its hash. Open the application on the Samsung and show the four-step account setup, explicit role selection, password visibility control and fixed primary action. The 709-check campaign records these interface behaviours, including registration text at 200%.
2. Show role-specific navigation and grouped Settings. The caregiver has no wearable connection control; the connection page is inside authenticated Settings. The adult's source-defined wearable guide separates preparation, pairing, selection and recent-sample verification.
3. Run fixed Normal, Warning and Critical fixtures through the authenticated classification API. Display the response class, model score and synthetic source separately from the care interface. Readings and rule episodes use their own storage endpoints.
4. Exercise a sustained rule condition with three eligible samples, inspect the stored episode and compare the two caregivers' private inboxes. The recorded HTTPS fixture establishes fanout and independent review at the server boundary.
5. Retry a sample after a lost response and compare its stored identity. Show invalid or missing inputs producing rejection or an unknown state. The backend comparison and policy tests establish these properties without assigning handset delivery latency to them.
6. Present the physical assembly in Figure 2, the source architecture and the recorded diagnostic aggregate. Explain which result comes from an image, a serial observation, a server test or a native interface check.

Table 10. Evaluation domains and the evidence supporting each claim.

Domain	Recorded evidence and interpretation	Assessment relationship
Android interface	Installed code 13; 709 native checks; registration at text scale 2.0; role and Settings navigation	Mobile functionality and programming
Classification	Two trained models, held out matrices and authenticated server inference; scores describe synthetic classes	AI and REST
Persistence and alerts	88 Python tests, 17 loopback HTTPS checks, 20 matched retry pairs and 20 episode trials	Data integration and testing
Hardware assembly	Original team photograph and 25-message diagnostic aggregate; image and runtime observation have distinct scope	Architecture and demonstration
Development trace	Team roster, four baseline commits and input hashes for the local release	Configuration and documentation

The care Activity presents readings, alerts and settings; the history endpoint and chart component are separate from its current navigation, and classification scores are exposed by the server interface. The measured results concern these explicit system boundaries. They establish neither clinical efficacy nor a medical safety certification, and they do not turn a synthetic class score into a disease probability.